DATA STRUCTURES AND ALGORITHMS

Assessment Tool 1

Name: Keerthika Devi G **Course code: CSA0305**

Register Number: 192524336 **Date: 22-10-2026**

1. What is Function Call Overhead in Recursion?

Definition: Function call overhead in recursion is the extra time and memory needed whenever a function calls itself.

Explanation: 1. Each recursive call creates a new function invocation.
2. The system stores parameters, local variables and the return address in the call stack.
3. Saving and restoring function states requires additional execution time.
4. Multiple calls remain on the stack until the base case is reached and the calls return.
5. Too many recursive calls may cause stack overflow.

Example (C):

```
int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}
```

Conclusion: Recursive function calls can increase execution time and stack-memory usage. Iteration may reduce this overhead in some problems.

2. Analyze the Performance of Linear Search vs Binary Search

Linear search checks each element sequentially. Binary search repeatedly halves the search interval and requires sorted data.

Feature	Linear Search	Binary Search
Method	Checks elements one by one	Repeatedly divides the search range in half
Data requirement	Sorted or unsorted data	Data must be sorted
Best-case time	O(1)	O(1)
Average-case time	O(n)	O(log n)
Worst-case time	O(n)	O(log n)
Space (iterative)	O(1)	O(1)
Suitable for	Small or unsorted lists	Large, sorted arrays

Example: Array = [10, 20, 30, 40, 50, 60, 70]; search key = 60.
Linear search compares 10, 20, 30, 40, 50 and then 60: 6 comparisons.
Binary search checks 40, then 60: 2 comparisons.

C Program – Linear Search

```
#include <stdio.h>

int main(void) {
    int a[] = {10, 20, 30, 40, 50};
    int n = 5, key = 40, i;

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            printf("Element found at position %d\n", i + 1);
            return 0;
        }
    }

    printf("Element not found\n");
    return 0;
}
```

Sample output: Element found at position 4

C Program – Binary Search

```
#include <stdio.h>

int main(void) {
    int a[] = {10, 20, 30, 40, 50};
    int low = 0, high = 4, mid, key = 40;

    while (low <= high) {
        mid = low + (high - low) / 2;

        if (a[mid] == key) {
            printf("Element found at position %d\n", mid + 1);
            return 0;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    printf("Element not found\n");
    return 0;
}
```

Sample output: Element found at position 4

Conclusion: Linear search is simple and works on unsorted data. Binary search is faster for large, sorted arrays because its worst-case time complexity is O(log n), compared with O(n) for linear search.

3. Implement Recursive Sum of Digits

Aim: To find the sum of the digits of a number using recursion.

Algorithm: 1. Read the number. 2. If n is 0, return 0. 3. Add the last digit (n % 10) to sumDigits(n / 10). 4. Display the result.

C Program

```c
#include <stdio.h>

int sumDigits(int n) {
    if (n == 0)
        return 0;

    return (n % 10) + sumDigits(n / 10);
}

int main(void) {
    int n;

    printf("Enter a non-negative integer: ");
    scanf("%d", &n);

    if (n < 0)
        n = -n;

    printf("Sum of digits = %d\n", sumDigits(n));
    return 0;
}
```

Sample input: 1234

Sample output: Sum of digits = 10

Result: The sum of the digits is calculated successfully using recursion.